

Project 8: ELK Stack SIEM - Clean Implementation Guide

Based on: Project 8 Complete Report & Addendum
Document Type: Step-by-Step Implementation Guide (Success Path Only)
Version: 1.0
Author: Derived from Prageeth Panicker's implementation
Estimated Time: 12-16 hours (clean path, excluding troubleshooting)

Document Purpose

This guide provides a **clean implementation path** for deploying ELK Stack SIEM with three integrated data sources:

- **Wazuh** (host-based security alerts)
- **Zeek** (network traffic analysis)
- **pfSense** (firewall logs)

Note: This extracts the **successful configurations** from the original 28-hour implementation, removing the troubleshooting journey to provide a direct path to success.

Table of Contents

1. [Prerequisites](#)
2. [Architecture Overview](#)
3. [Phase 1: ELK Stack Installation](#)
4. [Phase 2: Wazuh Integration](#)
5. [Phase 3: Zeek Integration](#)
6. [Phase 4: pfSense Integration](#)
7. [Phase 5: Dashboard Creation](#)
8. [Phase 6: Correlation Rules & Alerting](#)
9. [Validation & Testing](#)
10. [Troubleshooting Quick Reference](#)
11. [Operational Procedures](#)
12. [Appendix: Complete Configuration Files](#)

Prerequisites

Required Infrastructure (from Projects 1-7)

- ✓ **Project 7: Wazuh Manager**
 - Location: 192.168.30.10
 - Status: Operational with 8 agents
 - Alert file: /var/ossec/logs/alerts/alerts.json
- ✓ **Project 5: Zeek Network Monitor**
 - Location: 192.168.30.80
 - Status: Operational with cluster running
 - Logs: /opt/zeek/logs/current/conn.log

✔ Projects 1-2: pfSense Firewalls

- pfSense A: MAC-MINI (192.168.10.x networks)
- pfSense B: VM (192.168.10.5)
- Status: Operational with syslog capability

Hardware Requirements

ELK Stack VM:

- **RAM:** 8GB minimum (16GB recommended)
- **CPU:** 4 cores minimum
- **Disk:** 50GB minimum (100GB recommended)
- **OS:** Ubuntu 20.04+ LTS
- **Network:** Access to all data sources

Software Versions



Elasticsearch: 7.17.29

Logstash: 7.17.29

Kibana: 7.17.29

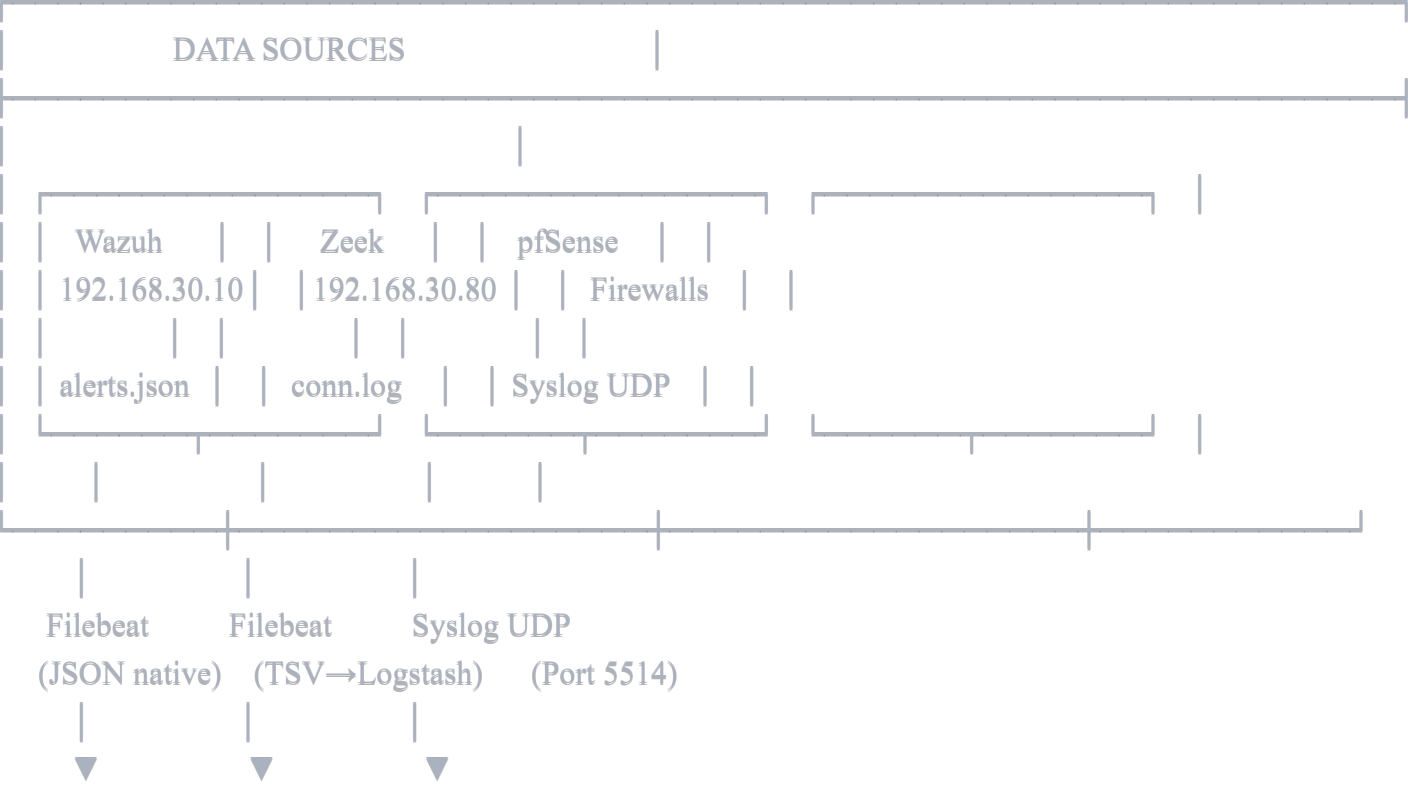
Filebeat: 7.17.29

Network Requirements

- **Management access:** SSH to all systems
- **Network connectivity:** ELK VM can reach:
 - 192.168.30.10 (Wazuh Manager)
 - 192.168.30.80 (Zeek Monitor)
 - 192.168.10.x (pfSense firewalls)
- **Firewall rules:** Allow inbound to ELK VM:
 - TCP 9200 (Elasticsearch)
 - TCP 5601 (Kibana)
 - TCP 5045 (Logstash beats input)
 - UDP 5514 (Logstash syslog input)

Architecture Overview





Access: <http://192.168.30.20:5601>

Credentials: elastic / SecureElastic

Phase 1: ELK Stack Installation

Time Estimate: 2-3 hours

Step 1.1: System Preparation



bash

Update system

```
sudo apt update && sudo apt upgrade -y
```

Install required packages

```
sudo apt install -y apt-transport-https curl gnupg2 software-properties-common
```

Install Java (required for Elasticsearch/Logstash)

```
sudo apt install -y openjdk-11-jdk
```

Verify Java installation

```
java -version
```

Step 1.2: Add Elastic Repository



bash

Import Elastic GPG key

```
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo apt-key add -
```

Add Elastic 7.x repository

```
echo "deb https://artifacts.elastic.co/packages/7.x/apt stable main" | sudo tee /etc/apt/sources.list.d/elastic-7.x.list
```

Update package list

```
sudo apt update
```

Step 1.3: Install Elasticsearch



bash

Install Elasticsearch

`sudo apt install -y elasticsearch`

Configure Elasticsearch

`sudo nano /etc/elasticsearch/elasticsearch.yml`

Add/modify these lines:



yaml

`cluster.name: elk-siem-cluster`

`node.name: elk-siem-node-1`

`network.host: 192.168.30.20`

`http.port: 9200`

`discovery.type: single-node`

X-Pack Security

`xpack.security.enabled: true`

`xpack.security.authc.api_key.enabled: true`

Start Elasticsearch:



bash

Reload systemd

```
sudo systemctl daemon-reload
```

Enable and start Elasticsearch

```
sudo systemctl enable elasticsearch
```

```
sudo systemctl start elasticsearch
```

Verify status

```
sudo systemctl status elasticsearch
```

Wait 30 seconds for startup

```
sleep 30
```

Test Elasticsearch (should prompt for authentication - expected)

```
curl -X GET "localhost:9200"
```

Step 1.4: Set Elasticsearch Passwords



bash

Navigate to Elasticsearch directory

```
cd /usr/share/elasticsearch
```

Set passwords interactively

```
sudo bin/elasticsearch-setup-passwords interactive
```

Set these passwords (use: SecureElastic for all or your own secure passwords):

- elastic (superuser)
- kibana_system
- logstash_system
- beats_system
- apm_system
- remote_monitoring_user

⚠ IMPORTANT: Record these passwords securely. You'll need them for configuration.

Test authentication:



bash

```
curl -u elastic:SecureElastic -X GET "http://localhost:9200"
```

Expected output:



json

```
{
  "name" : "elk-siem-node-1",
  "cluster_name" : "elk-siem-cluster",
  "version" : {
    "number" : "7.17.29"
  }
}
```

Step 1.5: Install Kibana



bash

```
# Install Kibana
sudo apt install -y kibana

# Configure Kibana
sudo nano /etc/kibana/kibana.yml
```

Add/modify these lines:



yaml

```
server.port: 5601
server.host: "192.168.30.20"
server.name: "elk-siem-kibana"

elasticsearch.hosts: ["http://192.168.30.20:9200"]
elasticsearch.username: "elastic"
elasticsearch.password: "SecureElastic"

# Encryption keys for alerting (CRITICAL - generate unique 32+ character keys)
xpack.encryptedSavedObjects.encryptionKey: "min-32-byte-encryption-key-abcd1234"
xpack.reporting.encryptionKey: "min-32-byte-reporting-key-security-9876"
xpack.security.encryptionKey: "min-32-byte-security-key-monitoring-5432"
```

⚠ SECURITY NOTE: Generate your own encryption keys:



bash

```
# Generate random 32-character keys
openssl rand -base64 32
```

Start Kibana:



bash

```
# Enable and start Kibana
sudo systemctl enable kibana
sudo systemctl start kibana

# Verify status
sudo systemctl status kibana

# Wait 60 seconds for startup
sleep 60
```

Access Kibana:

- URL: http://192.168.30.20:5601
- Username: elastic
- Password: SecureElastic

 **Checkpoint:** You should see the Kibana login page and be able to log in.

Step 1.6: Install Logstash



bash

Install Logstash

```
sudo apt install -y logstash
```

Create configuration directory (if not exists)

```
sudo mkdir -p /etc/logstash/conf.d
```

Set permissions

```
sudo chown -R logstash:logstash /usr/share/logstash/data
```

```
sudo chown -R logstash:logstash /etc/logstash
```

Enable Logstash (don't start yet - no configs)

```
sudo systemctl enable logstash
```

Note: We'll start Logstash after adding pipeline configurations.

Phase 2: Wazuh Integration

Time Estimate: 1-2 hours

Step 2.1: Install Filebeat on Wazuh Manager

On Wazuh Manager (192.168.30.10):



bash

Download Filebeat

```
cd /tmp
```

```
wget https://artifacts.elastic.co/downloads/beats/filebeat/filebeat-7.17.29-amd64.deb
```

Install Filebeat

```
sudo dpkg -i filebeat-7.17.29-amd64.deb
```

Step 2.2: Configure Filebeat for Wazuh Alerts



bash

Backup original config

```
sudo cp /etc/filebeat/filebeat.yml /etc/filebeat/filebeat.yml.backup
```

Create new configuration

```
sudo nano /etc/filebeat/filebeat.yml
```

Complete Filebeat configuration:



yaml

```
filebeat.inputs:
```

```
- type: log
```

```
  enabled: true
```

```
  paths:
```

```
    - /var/ossec/logs/alerts/alerts.json
```

```
  json.keys_under_root: true
```

```
  json.add_error_key: true
```

```
  json.overwrite_keys: true
```

```
output.logstash:
```

```
  hosts: ["192.168.30.20:5045"]
```

```
logging.level: info
```

```
logging.to_files: true
```

```
logging.files:
```

```
  path: /var/log/filebeat
```

```
  name: filebeat
```

```
  keepfiles: 7
```

⚠ **CRITICAL:** Verify the path is exactly `/var/ossec/logs/alerts/alerts.json` (not `alerst.json`)

Step 2.3: Configure Logstash for Wazuh

On ELK VM (192.168.30.20):



bash

Create Logstash pipeline for Wazuh

```
sudo nano /etc/logstash/conf.d/wazuh-alerts.conf
```

Wazuh Logstash pipeline:



ruby

```
input {
  beats {
    port => 5045
  }
}

filter {
  # No additional filtering needed - JSON already parsed by Filebeat
}

output {
  if [rule] {
    elasticsearch {
      hosts => ["http://localhost:9200"]
      user => "elastic"
      password => "SecureElastic"
      index => "filebeat-%{+YYYY.MM.dd}"
    }
  }
}
```

Step 2.4: Start Services

On ELK VM:



bash

```
# Start Logstash
sudo systemctl start logstash

# Verify Logstash started
sudo systemctl status logstash

# Check Logstash is listening on port 5045 (wait 30 seconds)
sleep 30
sudo netstat -tlnp | grep 5045
```

Expected output:



```
tcp6 0 0 :::5045 :::* LISTEN 12345/java
```

On Wazuh Manager:



bash

```
# Start Filebeat
sudo systemctl enable filebeat
sudo systemctl start filebeat

# Verify Filebeat status
sudo systemctl status filebeat
```

Step 2.5: Verify Wazuh Integration

Wait 2-3 minutes for data to flow, then check:

On ELK VM:



bash

```
# Check if filebeat index exists
curl -u elastic:SecureElastic "http://localhost:9200/_cat/indices/filebeat*?v"

# Check document count
curl -u elastic:SecureElastic "http://localhost:9200/filebeat-*/_count?pretty"
```

Expected output:



json

```
{  
  "count" : 150, // Or any number > 0  
  "_shards" : {  
    "total" : 1,  
    "successful" : 1,  
    "failed" : 0  
  }  
}
```

✅ **Checkpoint:** Document count should be increasing as Wazuh generates alerts.

If count is 0, check Filebeat logs:



bash

On Wazuh Manager

`sudo tail -50 /var/log/filebeat/filebeat`

Phase 3: Zeek Integration

Time Estimate: 2-3 hours

Step 3.1: Enable Promiscuous Mode on Zeek VM

On Zeek Monitor (192.168.30.80):

⚠️ **CRITICAL:** This step is essential for Zeek packet capture.



bash

Create systemd service for persistent promiscuous mode

`sudo nano /etc/systemd/system/promisc-mode.service`

Service configuration:



ini

[Unit]

Description=Set Network Interfaces to Promiscuous Mode

After=network.target

[Service]

Type=oneshot

ExecStart=/usr/sbin/ip link set em1 promisc on

ExecStart=/usr/sbin/ip link set em2 promisc on

RemainAfterExit=yes

[Install]

WantedBy=multi-user.target

Enable and start:



bash

```
sudo systemctl enable promisc-mode
```

```
sudo systemctl start promisc-mode
```

Verify promiscuous mode

```
ip link show em1 | grep PROMISC
```

```
ip link show em2 | grep PROMISC
```

Expected output:



```
<BROADCAST,MULTICAST,PROMISC,UP,LOWER_UP>
```

Step 3.2: Verify Zeek Cluster Status



bash

Check Zeek cluster status

```
sudo /opt/zeek/bin/zeekctl status
```

Expected: All workers and proxies running

If any workers show "crashed":



bash

```
sudo /opt/zeek/bin/zeekctl restart
```

Step 3.3: Install Filebeat on Zeek VM



bash

```
# Download Filebeat
cd /tmp
wget https://artifacts.elastic.co/downloads/beats/filebeat/filebeat-7.17.29-amd64.deb

# Install Filebeat
sudo dpkg -i filebeat-7.17.29-amd64.deb
```

Step 3.4: Configure Filebeat for Zeek



bash

```
# Backup original config
sudo cp /etc/filebeat/filebeat.yml /etc/filebeat/filebeat.yml.backup

# Create new configuration
sudo nano /etc/filebeat/filebeat.yml
```

Filebeat configuration:



yaml

```
filebeat.inputs:
- type: log
  enabled: true
  paths:
    - /opt/zeek/logs/current/conn.log
  fields:
    log_type: zeek-conn
  fields_under_root: true
```

```
output.logstash:
  hosts: ["192.168.30.20:5045"]
```

```
logging.level: info
logging.to_files: true
logging.files:
  path: /var/log/filebeat
  name: filebeat
  keepfiles: 7
```

Step 3.5: Configure Logstash for Zeek (CSV Filter)

On ELK VM (192.168.30.20):



bash

```
# Create Logstash pipeline for Zeek
sudo nano /etc/logstash/conf.d/zeek-conn.conf
```

Zeek Logstash pipeline (CSV filter approach):



ruby

```
input {
  beats {
    port => 5045
  }
}
```

```
filter {
  if [log_type] == "zeek-conn" {

    # Drop header lines (start with #)
    if [message] =~ /^#/ {
      drop { }
    }

    # Parse TSV with CSV filter (TAB-separated)
    csv {
      separator => " "
      columns => [
        "ts","uid","id.orig_h","id.orig_p","id.resp_h","id.resp_p",
        "proto","service","duration","orig_bytes","resp_bytes","conn_state",
        "local_orig","local_resp","missed_bytes","history","orig_pkts",
        "orig_ip_bytes","resp_pkts","resp_ip_bytes","tunnel_parents"
      ]
    }

    # Convert Unix timestamp to @timestamp
    date {
      match => ["ts", "UNIX"]
      target => "@timestamp"
    }

    # ECS Normalization - Map to Elastic Common Schema
    mutate {
      rename => {
        "[id.orig_h]" => "[source][ip]"
        "[id.orig_p]" => "[source][port]"
        "[id.resp_h]" => "[destination][ip]"
        "[id.resp_p]" => "[destination][port]"
      }
      remove_field => ["id"]
    }

    # Map proto to network.transport
    if [proto] in ["tcp","udp","icmp"] {
```

```
mutate {
  add_field => { "[network][transport]" => "%{proto}" }
}
}
```

Map service to ECS service.name

```
if [service] and [service] != "-" {
  mutate {
    rename => { "service" => "[service][name]" }
  }
}
```

Convert numeric fields

```
mutate {
  convert => {
    "[source][port]" => "integer"
    "[destination][port]" => "integer"
    "duration" => "float"
    "orig_bytes" => "integer"
    "resp_bytes" => "integer"
    "orig_pkts" => "integer"
    "resp_pkts" => "integer"
    "orig_ip_bytes" => "integer"
    "resp_ip_bytes" => "integer"
  }
}
```

Add tag for identification

```
mutate {
  add_tag => ["zeek"]
}
}
```

```
output {
  if "zeek" in [tags] {
    elasticsearch {
      hosts => ["http://localhost:9200"]
      user => "elastic"
      password => "SecureElastic"
      index => "zeek-conn3-%{+YYYY.MM.dd}"
    }
  }
}
```

```
}  
}
```

Step 3.6: Restart Services

On ELK VM:



bash

Restart Logstash to load new pipeline

```
sudo systemctl restart logstash
```

Verify Logstash restarted successfully

```
sudo systemctl status logstash
```

Wait 30 seconds

```
sleep 30
```

On Zeek VM:



bash

Start Filebeat

```
sudo systemctl enable filebeat
```

```
sudo systemctl start filebeat
```

Verify Filebeat status

```
sudo systemctl status filebeat
```

Step 3.7: Verify Zeek Integration

Wait 2-3 minutes for data to flow, then check:



bash

On ELK VM

```
curl -u elastic:SecureElastic "http://localhost:9200/_cat/indices/zeek*?v"
```

Check document count

```
curl -u elastic:SecureElastic "http://localhost:9200/zeek-conn3-*/_count?pretty"
```

Check field mappings (verify ECS normalization):



bash

```
curl -u elastic:SecureElastic "http://localhost:9200/zeek-conn3-*/_search?size=1&pretty" | grep -E "source|destination|"
```



Expected fields:

- source.ip
- source.port
- destination.ip
- destination.port
- network.transport
- service.name

☒ **Checkpoint:** Zeek documents should have properly mapped ECS fields.

Phase 4: pfSense Integration

Time Estimate: 1-2 hours

Step 4.1: Configure Logstash for pfSense Syslog

On ELK VM (192.168.30.20):



bash

Create Logstash pipeline for pfSense

```
sudo nano /etc/logstash/conf.d/pfsense-syslog.conf
```

pfSense Logstash pipeline (with grok pattern):



ruby

```
input {
  syslog {
    port => 5514
    type => "pfsense"
  }
}

filter {
  if [type] == "pfsense" {

    # Parse pfSense filterlog format with grok
    grok {
      match => {
        "message" => "%{SYSLOGTIMESTAMP:syslog_timestamp} %{SYSLOGHOST:syslog_host} filterlog: %{DATA:filterlog}"
      }
    }

    # Add metadata
    mutate {
      add_field => { "log_source" => "pfsense-direct" }
    }

    # Convert numeric fields
    mutate {
      convert => {
        "src_port" => "integer"
        "dest_port" => "integer"
      }
    }
  }
}

output {
  if [type] == "pfsense" {
    elasticsearch {
      hosts => ["http://localhost:9200"]
      user => "elastic"
      password => "SecureElastic"
      index => "pfsense-%{+YYYY.MM.dd}"
    }
  }
}
```

Step 4.2: Restart Logstash



bash

Restart Logstash to load pfSense pipeline

`sudo systemctl restart logstash`

Verify Logstash status

`sudo systemctl status logstash`

Wait 30 seconds

`sleep 30`

Verify syslog port 5514 is listening

`sudo netstat -ulnp | grep 5514`

Expected output:



`udp 0 0 0.0.0.0:5514 0.0.0.0:* 12345/java`

Step 4.3: Configure pfSense Remote Logging

On pfSense Web Interface (both pfSense A and B):

1. Navigate to: **Status** → **System Logs** → **Settings**
2. Scroll to **Remote Logging Options**
3. Configure:
 - **Enable Remote Logging:** ✓ Checked
 - **Source Address:** Select **LAN Interface** (NOT "Default")
 - **IP Protocol:** IPv4
 - **Remote log servers:** 192.168.30.20:5514
 - **Remote Syslog Contents:** Select:
 - ✓ Firewall Events
 - ✓ System Events
 - ✓ Authentication Events
4. Click **Save**

Step 4.4: Create pfSense Firewall Rule

On pfSense Web Interface:

1. Navigate to: **Firewall** → **Rules** → **LAN**
2. Click **Add** (up arrow) to add rule at top
3. Configure:
 - **Action:** Pass

- **Interface:** LAN
- **Address Family:** IPv4
- **Protocol:** UDP
- **Source:** LAN net
- **Destination:** Single host: 192.168.30.20
- **Destination Port Range:** Other - Custom: 5514 to 5514
- **Description:** "Allow pfSense syslog to ELK Stack"

4. Click **Save** → **Apply Changes**

Step 4.5: Verify pfSense Integration

On ELK VM:



bash

Monitor for incoming syslog traffic

```
sudo tcpdump -i any port 5514 -nn
```

You should see packets from pfSense IP addresses

Press Ctrl+C after seeing packets, then check Elasticsearch:



bash

Wait 2-3 minutes for logs to accumulate

```
sleep 180
```

Check if pfsense index exists

```
curl -u elastic:SecureElastic "http://localhost:9200/_cat/indices/pfsense*?v"
```

Check document count

```
curl -u elastic:SecureElastic "http://localhost:9200/pfsense-*/_count?pretty"
```

View sample document to verify parsing

```
curl -u elastic:SecureElastic "http://localhost:9200/pfsense-*/_search?size=1&pretty"
```

Verify parsed fields exist:

- action (block/pass)
- src_ip
- dest_ip
- src_port
- dest_port
- proto (tcp/udp/icmp)
- iface (em0/em1)

 **Checkpoint:** pfSense firewall logs should be flowing with parsed fields.

Phase 5: Dashboard Creation

Time Estimate: 2-3 hours

Step 5.1: Create Index Patterns in Kibana

Access Kibana: <http://192.168.30.20:5601>

Create Wazuh Index Pattern

1. Navigate to: **Stack Management** → **Index Patterns**
2. Click: **Create index pattern**
3. Configure:
 - **Index pattern name:** filebeat-*
 - **Time field:** @timestamp
4. Click: **Create index pattern**

Create Zeek Index Pattern

1. Click: **Create index pattern**
2. Configure:
 - **Index pattern name:** zeek-conn3-*
 - **Time field:** @timestamp
3. Click: **Create index pattern**

Create pfSense Index Pattern

1. Click: **Create index pattern**
2. Configure:
 - **Index pattern name:** pfsense-*
 - **Time field:** @timestamp
3. Click: **Create index pattern**

Step 5.2: Create Wazuh Security Dashboard

1. Navigate to: **Dashboard**
2. Click: **Create dashboard**
3. Click: **Create visualization**

Visualization 1: Top Wazuh Alert Rules

1. **Visualization type:** Vertical Bar
2. **Index pattern:** filebeat-*
3. **Metrics:**
 - Y-axis: Count
4. **Buckets:**
 - X-axis: Terms → rule.description.keyword → Size: 10
 - Split series: Terms → agent.name.keyword → Size: 5
5. **Time range:** Last 7 days
6. Click: **Save** → Name: "Top Wazuh Alert Rules"

Visualization 2: Alert Severity Over Time

1. **Visualization type:** Area
2. **Index pattern:** filebeat-*

3. **Metrics:**
 - Y-axis: Count
4. **Buckets:**
 - X-axis: Date Histogram → @timestamp
 - Split series: Terms → rule.level → Size: 10
5. Click: **Save** → Name: "Alert Severity Over Time"

Visualization 3: Alert Activity by Agent

1. **Visualization type:** Pie
2. **Index pattern:** filebeat-*
3. **Metrics:**
 - Slice size: Count
4. **Buckets:**
 - Split slices: Terms → agent.name.keyword → Size: 10
5. Click: **Save** → Name: "Alert Activity by Agent"

Save Dashboard: Click **Save** → Name: "Wazuh Security Overview"

Step 5.3: Create pfSense Firewall Dashboard

1. Navigate to: **Dashboard**
2. Click: **Create dashboard**

Visualization 1: Firewall Actions (Block vs Pass)

1. **Visualization type:** Pie
2. **Index pattern:** pfsense-*
3. **Metrics:** Count
4. **Buckets:** Split slices: Terms → action.keyword
5. Click: **Save** → Name: "Firewall Actions"

Visualization 2: Top Source IPs

1. **Visualization type:** Data Table
2. **Index pattern:** pfsense-*
3. **Metrics:** Count
4. **Buckets:** Split rows: Terms → src_ip.keyword → Size: 10
5. Click: **Save** → Name: "Top Source IPs"

Visualization 3: Top Destination Ports

1. **Visualization type:** Horizontal Bar
2. **Index pattern:** pfsense-*
3. **Metrics:** Count
4. **Buckets:** Y-axis: Terms → dest_port → Size: 10
5. Click: **Save** → Name: "Top Destination Ports"

Visualization 4: Traffic by Interface

1. **Visualization type:** Pie
2. **Index pattern:** pfsense-*
3. **Metrics:** Count
4. **Buckets:** Split slices: Terms → iface.keyword
5. Click: **Save** → Name: "Traffic by Interface"

Visualization 5: Blocked Traffic Timeline

1. **Visualization type:** Line
2. **Index pattern:** pfsense-*
3. **Filters:** action: block
4. **Metrics:** Count
5. **Buckets:** X-axis: Date Histogram → @timestamp
6. Click: **Save** → Name: "Blocked Traffic Timeline"

Save Dashboard: Click **Save** → Name: "pfSense Firewall Events"

Step 5.4: Create Zeek Network Activity Dashboard

1. Navigate to: **Dashboard**
2. Click: **Create dashboard**

Visualization 1: Top Source IPs by Connection Count

1. **Visualization type:** Data Table
2. **Index pattern:** zeek-conn3-*
3. **Metrics:** Count
4. **Buckets:** Split rows: Terms → source.ip.keyword → Size: 10
5. Click: **Save** → Name: "Top Source IPs"

Visualization 2: Top Destination IPs

1. **Visualization type:** Data Table
2. **Index pattern:** zeek-conn3-*
3. **Metrics:** Count
4. **Buckets:** Split rows: Terms → destination.ip.keyword → Size: 10
5. Click: **Save** → Name: "Top Destination IPs"

Visualization 3: Network Protocols Distribution

1. **Visualization type:** Pie
2. **Index pattern:** zeek-conn3-*
3. **Metrics:** Count
4. **Buckets:** Split slices: Terms → network.transport.keyword
5. Click: **Save** → Name: "Network Protocols"

Visualization 4: Connection States

1. **Visualization type:** Bar
2. **Index pattern:** zeek-conn3-*
3. **Metrics:** Count
4. **Buckets:** X-axis: Terms → conn_state.keyword
5. Click: **Save** → Name: "Connection States"

Visualization 5: Service Distribution

1. **Visualization type:** Pie
2. **Index pattern:** zeek-conn3-*
3. **Metrics:** Count
4. **Buckets:** Split slices: Terms → service.name.keyword → Size: 10
5. Click: **Save** → Name: "Service Distribution"

Save Dashboard: Click **Save** → Name: "Zeek Network Activity"



Checkpoint: You should now have 3 operational dashboards with real-time data.

Phase 6: Correlation Rules & Alerting

Time Estimate: 1-2 hours

Step 6.1: Verify Alerting Prerequisites

Check encryption keys are configured:



bash

```
# On ELK VM
sudo grep -n "encryptionKey" /etc/kibana/kibana.yml | grep -v "^#"
```

Expected output:



```
117:xpack.encryptedSavedObjects.encryptionKey: "min-32-byte-encryption-key-abcd1234"
118:xpack.reporting.encryptionKey: "min-32-byte-reporting-key-security-9876"
119:xpack.security.encryptionKey: "min-32-byte-security-key-monitoring-5432"
```

If not present, add them and restart Kibana:



bash

```
sudo systemctl restart kibana
sleep 60
```

Verify Elasticsearch API keys enabled:



bash

```
sudo grep "api_key.enabled" /etc/elasticsearch/elasticsearch.yml
```

Expected output:



```
xpack.security.authc.api_key.enabled: true
```

If not present, add it and restart Elasticsearch:



bash

```
sudo systemctl restart elasticsearch
```

```
sleep 30
```

Step 6.2: Create Index Connector

In Kibana UI:

1. Navigate to: **Stack Management** → **Rules and Connectors**
2. Click: **Connectors** tab
3. Click: **Create connector**
4. Select: **Index**
5. Configure:
 - **Connector name:** Security Alerts Connector
 - **Index name:** security-alerts
 - **Description:** "Persistent storage for correlation rule alerts"
6. Click: **Save**

✅ **Checkpoint:** Connector should appear in connectors list.

Step 6.3: Create Correlation Rule - Brute Force Detection

1. Navigate to: **Stack Management** → **Rules and Connectors**
2. Click: **Rules** tab
3. Click: **Create rule**
4. Configure:

Rule Details:

- **Name:** Brute Force Detection - High Alert Volume
- **Check every:** 1 minute
- **Notify:** Every time alert status changes

Rule Type:

- Select: **Elasticsearch query**

Index:

- Select: filebeat-*

Time field:

- Select: @timestamp

Define the threshold:

- **When:** count()

- **Is:** ABOVE
- **Threshold:** 5
- **For the last:** 1 minute
- **Group alerts by:** Top 5 'agent.name.keyword'

Actions:

Action 1: Index Connector

1. Click: **Add action**
2. Select: **Security Alerts Connector**
3. **Document to index:**



json

```
{
  "rule_name": "{{ruleName}}",
  "alert_time": "{{date}}",
  "threshold_value": "{{context.value}}",
  "conditions_met": "{{context.conditions}}",
  "alert_id": "{{alertId}}"
}
```

Action 2: Server Log

1. Click: **Add action**
2. Select: **Server log**
3. **Message:**



```
SECURITY ALERT: {{ruleName}} triggered at {{date}}. Alert count: {{context.value}}
```

4. Click: **Save**

☒ **Checkpoint:** Rule should appear in rules list with status "Pending" (pink) initially.

Step 6.4: Verify Correlation Rule

Wait 1-2 minutes for first execution.

Check rule status in Kibana:

1. Navigate to: **Stack Management → Rules and Connectors → Rules**
2. Verify rule shows "Ok" (green) or "Active" (red if threshold exceeded)

Check alert index:



bash

```
# On ELK VM (after first alert triggers)
curl -u elastic:SecureElastic "http://localhost:9200/security-alerts/_search?size=1&sort=alert_time:desc&pretty"
```

Check Kibana logs:



bash

```
sudo tail -f /var/log/kibana/kibana.log | grep "SECURITY ALERT"
```

☒ **Checkpoint:** When rule triggers (>5 alerts in 1 minute), you should see:

- Alert document in security-alerts index
- "SECURITY ALERT" message in Kibana logs

Validation & Testing

Test 1: Generate Wazuh Alerts

On any Wazuh agent system:



bash

```
# Generate authentication events
sudo su - root
exit
```

```
# Generate file creation events
echo "test" > /tmp/test-file.txt
rm /tmp/test-file.txt
```

```
# Generate network activity
ping -c 5 google.com
```

Verify in Kibana:

1. Navigate to: **Discover**
2. Select: filebeat-* index pattern
3. Time range: Last 15 minutes
4. Search: agent.name: "YOUR-AGENT-NAME"

Expected: New alert documents appear within 1-2 minutes.

Test 2: Generate Zeek Network Traffic

From any system in monitored network:



bash

```
# Generate DNS queries
nslookup google.com

# Generate HTTP traffic
curl http://example.com

# Generate HTTPS traffic
curl https://www.google.com
```

Verify in Kibana:

- 1. Navigate to: **Discover**
- 2. Select: zeek-conn3-* index pattern
- 3. Time range: Last 15 minutes

Expected: New connection documents with ECS fields (source.ip, destination.ip, network.transport).

Test 3: Generate pfSense Firewall Events

From internal network:



bash

```
# Generate allowed traffic
ping 8.8.8.8

# Generate blocked traffic (if you have block rules)
# Try accessing blocked port or IP
```

Verify in Kibana:

- 1. Navigate to: **Discover**
- 2. Select: pfsense-* index pattern
- 3. Time range: Last 15 minutes

Expected: Firewall logs with action (pass/block), src_ip, dest_ip fields.

Test 4: Trigger Correlation Rule

Generate high alert volume:

On multiple Wazuh agents or single agent:



```
bash

# Run loop to generate multiple events quickly
for i in {1..10}; do
  sudo su - root -c "exit"
  sleep 2
done
```

Verify rule triggered:



```
bash

# Check security-alerts index
curl -u elastic:SecureElastic "http://localhost:9200/security-alerts/_count?pretty"

# Check Kibana logs
sudo grep "SECURITY ALERT" /var/log/kibana/kibana.log | tail -5
```

Expected: Alert document created, log entry visible.

Troubleshooting Quick Reference

Issue: No Wazuh alerts flowing

Symptoms: filebeat-* index has 0 documents

Checks:



bash

On Wazuh Manager

```
sudo systemctl status filebeat
```

```
sudo tail -50 /var/log/filebeat/filebeat
```

Verify path is correct (not "alerst.json")

```
sudo grep "paths:" /etc/filebeat/filebeat.yml
```

Test connectivity to ELK

```
telnet 192.168.30.20 5045
```

On ELK VM

```
sudo netstat -tlnp | grep 5045
```

```
sudo tail -50 /var/log/logstash/logstash-plain.log
```

Common fixes:

1. Fix typo in path: /var/ossec/logs/alerts/alerts.json
 2. Clear Filebeat registry: `sudo rm -rf /var/lib/filebeat/registry`
 3. Restart Filebeat: `sudo systemctl restart filebeat`
-

Issue: Zeek logs not parsed (no ECS fields)

Symptoms: Documents in zeek-conn3-* but no source.ip or destination.ip fields

Checks:



bash

Verify Zeek cluster is running

```
sudo /opt/zeek/bin/zeekctl status
```

Verify promiscuous mode

```
ip link show em1 | grep PROMISC
```

Check Logstash pipeline

```
sudo cat /etc/logstash/conf.d/zeek-conn.conf | grep csv
```

Common fixes:

1. Enable promiscuous mode (see Step 3.1)
 2. Restart Zeek cluster: `sudo /opt/zeek/bin/zeekctl restart`
 3. Verify CSV filter in Logstash config
 4. Restart Logstash: `sudo systemctl restart logstash`
-

Issue: pfSense logs not arriving

Symptoms: pfsense-* index doesn't exist or has 0 documents

Checks:



bash

```
# On ELK VM
sudo netstat -ulnp | grep 5514
sudo tcpdump -i any port 5514 -nn

# Check Logstash logs
sudo tail -50 /var/log/logstash/logstash-plain.log | grep -i pfsense
```

Common fixes:

- 1. Verify pfSense firewall rule allows UDP 5514 to ELK VM
- 2. Change pfSense syslog source from "Default" to "LAN Interface"
- 3. Verify Remote Logging enabled in pfSense
- 4. Check Logstash password is not placeholder

Issue: Kibana won't start

Symptoms: Cannot access Kibana UI

Checks:



bash

```
sudo systemctl status kibana
sudo tail -100 /var/log/kibana/kibana.log
```

Common fixes:

- 1. Verify encryption keys in /etc/kibana/kibana.yml
- 2. Verify Elasticsearch password is correct
- 3. Check Elasticsearch is running: sudo systemctl status elasticsearch
- 4. Restart Kibana: sudo systemctl restart kibana

Issue: Correlation rule not triggering

Symptoms: Rule status stays "Ok", no alerts in security-alerts index

Checks:



bash

```
# Verify encryption keys configured
sudo grep encryptionKey /etc/kibana/kibana.yml

# Verify API keys enabled
sudo grep api_key /etc/elasticsearch/elasticsearch.yml

# Check rule execution in Kibana logs
sudo tail -f /var/log/kibana/kibana.log | grep -i "brute\|alert"
```

Common fixes:

- 1. Add encryption keys to /etc/kibana/kibana.yml (lines 117-119)
- 2. Add xpack.security.authc.api_key.enabled: true to Elasticsearch
- 3. Restart both services
- 4. Manually trigger rule by generating >5 alerts in 1 minute

Operational Procedures

Daily Monitoring Tasks

Task 1: Check Service Status



bash

```
#!/bin/bash
```

```
# File: /opt/scripts/elk-health-check.sh
```

```
echo "=== ELK Stack Health Check ==="
```

```
echo ""
```

```
echo "Elasticsearch:"
```

```
sudo systemctl status elasticsearch | grep "Active:"
```

```
curl -u elastic:SecureElastic -s "http://localhost:9200/_cluster/health?pretty" | grep status
```

```
echo ""
```

```
echo "Kibana:"
```

```
sudo systemctl status kibana | grep "Active:"
```

```
echo ""
```

```
echo "Logstash:"
```

```
sudo systemctl status logstash | grep "Active:"
```

```
echo ""
```

```
echo "=== Data Sources ==="
```

```
curl -u elastic:SecureElastic -s "http://localhost:9200/_cat/indices/*?v" | grep -E "filebeat|zeek|pfsense"
```

```
echo ""
```

```
echo "=== Document Counts ==="
```

```
echo "Wazuh alerts:"
```

```
curl -u elastic:SecureElastic -s "http://localhost:9200/filebeat-*/_count" | grep count
```

```
echo "Zeek network logs:"
```

```
curl -u elastic:SecureElastic -s "http://localhost:9200/zeek-conn3-*/_count" | grep count
```

```
echo "pfSense firewall logs:"
```

```
curl -u elastic:SecureElastic -s "http://localhost:9200/pfsense-*/_count" | grep count
```

Run daily:



bash

```
sudo bash /opt/scripts/elk-health-check.sh
```

Task 2: Check Correlation Rules Status

In Kibana UI:

1. Navigate to: **Stack Management** → **Rules and Connectors** → **Rules**
 2. Review status column:
 - **Ok** (green): Rule executed, threshold not met
 - **Active** (red): Rule triggered, alert generated
 - **Error** (red): Rule execution failed - investigate immediately
 3. Investigate any "Error" status rules
-

Task 3: Review Security Alerts



bash

```
# Check alert index for today's alerts
curl -u elastic:SecureElastic "http://localhost:9200/security-alerts/_search?size=100&sort=alert_time:desc&pretty"

# Or in Kibana UI:
# Navigate to Discover → Select security-alerts index pattern
```

Weekly Maintenance Tasks

Task 1: Backup Kibana Saved Objects



bash

```
#!/bin/bash
```

```
# File: /opt/scripts/backup-kibana-objects.sh
```

```
BACKUP_DIR="/backup/elk/kibana"
```

```
DATE=$(date +%Y%m%d)
```

```
KIBANA_URL="http://192.168.30.20:5601"
```

```
# Create backup directory
```

```
mkdir -p $BACKUP_DIR
```

```
# Export all saved objects
```

```
curl -u elastic:SecureElastic -X POST "$KIBANA_URL/api/saved_objects/_export" \
-H 'kbn-xsrf: true' \
-H 'Content-Type: application/json' \
-d '{"type":["dashboard","visualization","search","index-pattern","alert"]}' \
-o "$BACKUP_DIR/kibana-objects-$DATE.ndjson"
```

```
echo "Backup completed: $BACKUP_DIR/kibana-objects-$DATE.ndjson"
```

```
# Keep only last 4 weeks of backups
```

```
find $BACKUP_DIR -name "kibana-objects-*.ndjson" -mtime +28 -delete
```

Schedule with cron:



bash

```
# Run every Sunday at 2 AM
```

```
0 2 * * 0 /opt/scripts/backup-kibana-objects.sh
```

Task 2: Check Disk Usage



bash

```
# Check Elasticsearch disk usage
```

```
df -h /var/lib/elasticsearch
```

```
# Check index sizes
```

```
curl -u elastic:SecureElastic "http://localhost:9200/_cat/indices?v&h=index,store.size&s=store.size:desc"
```

Alert thresholds:

- **Warning:** >70% disk usage
- **Critical:** >85% disk usage

Actions if critical:

1. Delete old indices manually
2. Implement Index Lifecycle Management (ILM)

Backup Procedures

Full Configuration Backup



bash

```
#!/bin/bash
```

```
# File: /opt/scripts/elk-full-backup.sh
```

```
BACKUP_ROOT="/backup/elk"
```

```
DATE=$(date +%Y%m%d-%H%M%S)
```

```
BACKUP_DIR="$BACKUP_ROOT/backup-$DATE"
```

```
mkdir -p $BACKUP_DIR
```

```
echo "Backing up ELK Stack configuration..."
```

```
# Elasticsearch
```

```
mkdir -p $BACKUP_DIR/elasticsearch
```

```
sudo cp /etc/elasticsearch/elasticsearch.yml $BACKUP_DIR/elasticsearch/
```

```
sudo cp /etc/default/elasticsearch $BACKUP_DIR/elasticsearch/ 2>/dev/null
```

```
# Kibana
```

```
mkdir -p $BACKUP_DIR/kibana
```

```
sudo cp /etc/kibana/kibana.yml $BACKUP_DIR/kibana/
```

```
# Logstash
```

```
mkdir -p $BACKUP_DIR/logstash
```

```
sudo cp -r /etc/logstash/conf.d/ $BACKUP_DIR/logstash/
```

```
# Filebeat configurations (if local)
```

```
if [ -d /etc/filebeat ]; then
```

```
    mkdir -p $BACKUP_DIR/filebeat
```

```
    sudo cp /etc/filebeat/filebeat.yml $BACKUP_DIR/filebeat/
```

```
fi
```

```
echo "Backup completed: $BACKUP_DIR"
```

Disaster Recovery

Restore from Backup



bash

Restore Elasticsearch configuration

```
sudo cp /backup/elk/backup-YYYYMMDD/elasticsearch/elasticsearch.yml /etc/elasticsearch/  
sudo systemctl restart elasticsearch
```

Restore Kibana configuration

```
sudo cp /backup/elk/backup-YYYYMMDD/kibana/kibana.yml /etc/kibana/  
sudo systemctl restart kibana
```

Restore Logstash pipelines

```
sudo cp -r /backup/elk/backup-YYYYMMDD/logstash/conf.d/* /etc/logstash/conf.d/  
sudo systemctl restart logstash
```

Restore Kibana saved objects

In Kibana UI:

Stack Management → Saved Objects → Import

Select backed-up NDJSON file

Appendix: Complete Configuration Files

A. Elasticsearch Configuration

File: /etc/elasticsearch/elasticsearch.yml



yaml

Cluster

cluster.name: elk-siem-cluster

node.name: elk-siem-node-1

Network

network.host: 192.168.30.20

http.port: 9200

Discovery

discovery.type: single-node

Security

xpack.security.enabled: true

xpack.security.authc.api_key.enabled: true

Paths

path.data: /var/lib/elasticsearch

path.logs: /var/log/elasticsearch

B. Kibana Configuration

File: /etc/kibana/kibana.yml



yaml

Server

server.port: 5601

server.host: "192.168.30.20"

server.name: "elk-siem-kibana"

Elasticsearch

elasticsearch.hosts: ["http://192.168.30.20:9200"]

elasticsearch.username: "elastic"

elasticsearch.password: "SecureElastic"

Encryption keys (GENERATE YOUR OWN)

xpack.encryptedSavedObjects.encryptionKey: "min-32-byte-encryption-key-abcd1234"

xpack.reporting.encryptionKey: "min-32-byte-reporting-key-security-9876"

xpack.security.encryptionKey: "min-32-byte-security-key-monitoring-5432"

Logging

logging.dest: /var/log/kibana/kibana.log

C. Filebeat Configuration (Wazuh Manager)

File: /etc/filebeat/filebeat.yml on Wazuh Manager



yaml

```
filebeat.inputs:
- type: log
  enabled: true
  paths:
    - /var/ossec/logs/alerts/alerts.json
  json.keys_under_root: true
  json.add_error_key: true
  json.overwrite_keys: true
```

```
output.logstash:
  hosts: ["192.168.30.20:5045"]
```

```
logging.level: info
logging.to_files: true
logging.files:
  path: /var/log/filebeat
  name: filebeat
  keepfiles: 7
```

D. Filebeat Configuration (Zeek Monitor)

File: /etc/filebeat/filebeat.yml on Zeek VM



yaml

```
filebeat.inputs:
- type: log
  enabled: true
  paths:
    - /opt/zeek/logs/current/conn.log
  fields:
    log_type: zeek-conn
  fields_under_root: true
```

```
output.logstash:
  hosts: ["192.168.30.20:5045"]
```

```
logging.level: info
logging.to_files: true
logging.files:
  path: /var/log/filebeat
  name: filebeat
  keepfiles: 7
```

E. Logstash Pipeline - Wazuh

File: /etc/logstash/conf.d/wazuh-alerts.conf



ruby

```

input {
  beats {
    port => 5045
  }
}

filter {
  # JSON already parsed by Filebeat
}

output {
  if [rule] {
    elasticsearch {
      hosts => ["http://localhost:9200"]
      user => "elastic"
      password => "SecureElastic"
      index => "filebeat-%{+YYYY.MM.dd}"
    }
  }
}

```

F. Logstash Pipeline - Zeek

File: /etc/logstash/conf.d/zeek-conn.conf



ruby

```
input {
  beats {
    port => 5045
  }
}
```

```
filter {
  if [log_type] == "zeek-conn" {

    if [message] =~ /^#/ {
      drop { }
    }

    csv {
      separator => " "
      columns => [
        "ts","uid","id.orig_h","id.orig_p","id.resp_h","id.resp_p",
        "proto","service","duration","orig_bytes","resp_bytes","conn_state",
        "local_orig","local_resp","missed_bytes","history","orig_pkts",
        "orig_ip_bytes","resp_pkts","resp_ip_bytes","tunnel_parents"
      ]
    }

    date {
      match => ["ts", "UNIX"]
      target => "@timestamp"
    }

    mutate {
      rename => {
        "[id.orig_h]" => "[source][ip]"
        "[id.orig_p]" => "[source][port]"
        "[id.resp_h]" => "[destination][ip]"
        "[id.resp_p]" => "[destination][port]"
      }
      remove_field => ["id"]
    }

    if [proto] in ["tcp","udp","icmp"] {
      mutate {
        add_field => { "[network][transport]" => "%{proto}" }
      }
    }
  }
}
```

```

if [service] and [service] != "-" {
  mutate {
    rename => { "service" => "[service][name]" }
  }
}

```

```

mutate {
  convert => {
    "[source][port]" => "integer"
    "[destination][port]" => "integer"
    "duration" => "float"
    "orig_bytes" => "integer"
    "resp_bytes" => "integer"
    "orig_pkts" => "integer"
    "resp_pkts" => "integer"
  }
}

```

```

mutate {
  add_tag => ["zeek"]
}
}

```

```

output {
  if "zeek" in [tags] {
    elasticsearch {
      hosts => ["http://localhost:9200"]
      user => "elastic"
      password => "SecureElastic"
      index => "zeek-conn3-%{+YYYY.MM.dd}"
    }
  }
}

```

G. Logstash Pipeline - pfSense

File: /etc/logstash/conf.d/pfsense-syslog.conf



ruby

```
input {
  syslog {
    port => 5514
    type => "pfsense"
  }
}

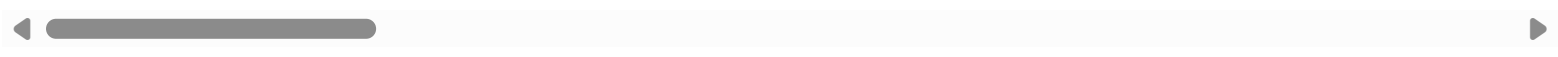
filter {
  if [type] == "pfsense" {

    grok {
      match => {
        "message" => "%{SYSLOGTIMESTAMP:syslog_timestamp} %{SYSLOGHOST:syslog_host} filterlog: %{DATA:message}"
      }
    }

    mutate {
      add_field => { "log_source" => "pfsense-direct" }
    }

    mutate {
      convert => {
        "src_port" => "integer"
        "dest_port" => "integer"
      }
    }
  }
}

output {
  if [type] == "pfsense" {
    elasticsearch {
      hosts => ["http://localhost:9200"]
      user => "elastic"
      password => "SecureElastic"
      index => "pfsense-%{+YYYY.MM.dd}"
    }
  }
}
```



H. Promiscuous Mode Service (Zeek VM)

File: /etc/systemd/system/promisc-mode.service



ini

[Unit]
Description=Set Network Interfaces to Promiscuous Mode
After=network.target

[Service]
Type=oneshot
ExecStart=/usr/sbin/ip link set em1 promisc on
ExecStart=/usr/sbin/ip link set em2 promisc on
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target

Enable:



bash

sudo systemctl enable promisc-mode
sudo systemctl start promisc-mode

Summary Checklist

Installation Checklist

- ☐ ELK Stack VM prepared (8GB RAM, 4 CPU, 50GB disk)
- ☐ Elasticsearch installed and running
- ☐ Kibana installed and accessible
- ☐ Logstash installed with pipelines configured
- ☐ Wazuh Filebeat installed and configured
- ☐ Zeek Filebeat installed and configured
- ☐ pfSense remote logging configured
- ☐ All index patterns created in Kibana
- ☐ Three dashboards created (Wazuh, Zeek, pfSense)
- ☐ Correlation rule created and active
- ☐ Automated alerting configured and tested

Validation Checklist

- ☐ filebeat-* index exists with documents (Wazuh)
- ☐ zeek-conn3-* index exists with ECS fields
- ☐ pfsense-* index exists with parsed fields
- ☐ Wazuh dashboard displays data
- ☐ Zeek dashboard displays data
- ☐ pfSense dashboard displays data
- ☐ Correlation rule status shows "Ok" or "Active"
- ☐ security-alerts index created (after first alert)
- ☐ Alert appears in Kibana logs

Operational Checklist

- ☐ Daily health check script configured
- ☐ Weekly backup script scheduled
- ☐ Disk usage monitoring in place
- ☐ Documentation accessible to team
- ☐ Credentials stored securely
- ☐ Disaster recovery procedure tested

Expected Results

After completing this implementation, you should have:

Infrastructure:

- ☒ ELK Stack operational with 3 components
- ☒ X-Pack security enabled with authentication
- ☒ 4,000+ security events indexed

Data Sources:

- ☒ Wazuh: 3,000+ alert documents
- ☒ Zeek: 300+ network connection documents with ECS fields
- ☒ pfSense: 500+ firewall log documents with parsed fields

Visualizations:

- ☒ 3 operational dashboards (15+ visualizations total)
- ☒ Real-time security event visibility
- ☒ Cross-source correlation capability

Automation:

- ☒ 1 correlation rule (brute force detection)
- ☒ Automated alerting to index + logs
- ☒ Foundation for additional rules

Operational Capability:

- ☒ Centralized SIEM platform
- ☒ Unified search across all security data
- ☒ Alert investigation workflows

-  Compliance framework support (MITRE ATT&CK, PCI DSS, HIPAA, NIST, GDPR)
-

Next Steps

After successful implementation:

1. **Create Additional Correlation Rules:**
 - Port scanning detection
 - Data exfiltration patterns
 - Failed authentication attempts
 - Suspicious network connections
 2. **Expand Data Sources:**
 - Additional Zeek logs (dns.log, http.log, ssl.log)
 - Windows Event Logs
 - Application logs
 - Cloud infrastructure logs
 3. **Implement Index Lifecycle Management:**
 - Automate old index deletion
 - Configure hot-warm-cold architecture
 - Optimize storage costs
 4. **Enhance Dashboards:**
 - Executive security overview
 - Threat actor TTPs
 - Compliance reporting
 - Performance metrics
 5. **Integrate with SOAR:**
 - Shuffle workflow automation
 - Automated incident response
 - Playbook development
-

Support Resources

Official Documentation:

- [Elasticsearch 7.17](#)
- [Logstash 7.17](#)
- [Kibana 7.17](#)
- [Filebeat 7.17](#)

Community:

- [Elastic Community Forums](#)
- [Wazuh Community](#)
- [Zeek Community](#)

Standards:

- [Elastic Common Schema \(ECS\)](#)
 - [MITRE ATT&CK Framework](#)
-

Document Version: 1.0

Based On: Project 8 Complete Report + Addendum

Success Path: Extracted from 28-hour implementation

Estimated Time: 12-16 hours (clean path)

Author: Implementation guide derived from Prageeth Panicker's work

 This implementation guide provides the direct path to success, avoiding the 24 hours of troubleshooting documented in the original report while preserving all working configurations and best practices.